

Tema 4.2 — Herencia

1) Idea clave: “A es-un B”

La herencia define una clase nueva (subclase) a partir de otra (superclase) y se interpreta como relación “A es-un B”. Ej.: Artillero es-un Soldado, Zapador es-un Soldado.

Implicaciones principales

Compatibilidad de tipos: un objeto de subtipo puede tratarse como su supertipo (puedes guardarlo en variables/arrays de Soldado).

Herencia de estado y comportamiento: la subclase hereda atributos y métodos definidos en la superclase (por ejemplo, el nombre y saludar()).

2) Constructores: cuántos se ejecutan y en qué orden

Al crear un objeto de una subclase se ejecutan los constructores de toda la jerarquía: primero el de la superclase y luego el de la subclase (de “arriba hacia abajo”).

super(...) en un constructor sirve para invocar explícitamente el constructor de la superclase y pasarle parámetros; debe ser la primera instrucción del constructor.

Si la superclase no tiene un constructor sin parámetros accesible, es obligatorio llamar a super(...) explícitamente (si no, el compilador intenta super() y falla).

3) Atributos private del padre: ¿existen en memoria en la subclase?

Sí: la instancia de la subclase contiene toda la parte de la superclase en memoria + su parte específica.

Pero private significa que no se puede acceder desde el código de la subclase: aunque exista, no puedes hacer this.nombre si nombre es private en Soldado.

Para usarlo, se recurre a métodos de la superclase (getters o métodos que lo utilicen como saludar()).

4) Extensibilidad por compatibilidad de tipos

Como los subtipos son compatibles con el supertipo, puedes añadir nuevos tipos (ej.: Medico extends Soldado) sin modificar el código cliente que trabaja con Soldado (por ejemplo, el bucle que recorre un array de Soldado y llama a saludar()).

Esto se relaciona con la idea de extender el sistema sin “romper” el código existente.

5) Referencias, upcasting, downcasting e instanceof

Es posible tener una referencia del supertipo apuntando a un objeto real de subtipo (Soldado s = new Artillero(...)).

Con esa referencia de supertipo, solo puedes invocar lo que el compilador conoce del tipo Soldado (no métodos exclusivos de Artillero sin conversión).

Upcasting

Convertir “hacia arriba”: tratar un subtipo como supertipo. Suele ser implícito y seguro.

Downcasting

Convertir “hacia abajo”: pasar de supertipo a subtipo para acceder a funcionalidad específica.

Antes de hacerlo, se usa instanceof para comprobar el tipo real y evitar ClassCastException.

6) protected (acceso protegido) y encapsulación

Protected permite acceso desde:

- *la propia clase

- *sus subclases (incluso en otros paquetes).

Está entre private (solo clase) y public (todos).

Se menciona que facilita reutilización en jerarquías, aunque reduce protección frente a private.

7) ¿Existe una clase base común para todo?

En Java sí: todas las clases heredan de Object, aunque no se escriba explícitamente.

Eso implica que métodos como toString(), equals() o hashCode() existen para cualquier objeto.

8) Herencia múltiple

Herencia múltiple = heredar de más de una superclase a la vez.

Java no permite herencia múltiple de clases (solo un extends), pero sí permite “algo parecido” mediante interfaces (una clase puede implementar varias).

9) Excepciones personalizadas como objetos + composición

En Java, las excepciones son objetos; se pueden crear excepciones propias. Una excepción no controlada extiende RuntimeException (no obliga a throws/captura).

Además, una excepción puede “transportar” datos componiéndose con otros objetos (por ejemplo, incluir un Usuario problemático) y también puede encadenar una causa mediante sobrecarga de constructores.

10) Herencia vs composición: por qué no usar herencia solo para reutilizar

Se indica que no debe usarse herencia solo por “reutilizar código” porque:

- *la herencia impone una relación fuerte “es-un” que debe tener sentido;
- *puede hacer que una subclase herede comportamientos que no le encajan y violar sustitución;
- *crea acoplamiento fuerte: cambios en la superclase pueden afectar a subclases.

La composición permite reutilizar delegando, con mayor flexibilidad y control.

11) “Favorecer composición frente a herencia”

La composición suele ser más modular y desacoplada: eliges qué partes reutilizas como componentes internos y puedes sustituirlos sin atarte a una jerarquía rígida.

12) “La herencia rompe la encapsulación”

La idea es que la subclase queda expuesta a detalles internos de la superclase y puede verse afectada por cambios internos aunque la interfaz pública no cambie; además, el uso de `protected` puede aumentar ese acoplamiento.

13) Dos modelos equivalentes: herencia vs composición (Persona/DatosPersonales)

Se propone modelar datos comunes (DNI, nombre) de dos maneras:

*Herencia: Estudiante y Trabajador “son” Persona. [universidad...epoint.com]

*Composición: ambos “tienen” DatosPersonales y lo reciben por constructor.